

Vel Tech Group of Institutions

Name: GOPICHAND KAKIVAI

Email: vtu29748@veltech.edu.in

Roll no: vtu29748

Phone: null

Branch: VTU

Department: CSE

Batch: 2028

Degree: B.Tech - Computer Science and Engineering

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS1L3

VTU_DAA_Week 9_COD

Attempt : 1

Total Mark : 40

Marks Obtained : 40

Section 1 : Coding

1. Problem Statement

Bob has a task to find a Hamiltonian cycle in a given undirected simple graph (no self-loops) represented as an adjacency matrix, where a value of 1 indicates an edge exists between two vertices and a value of 0 indicates no edge. A Hamiltonian cycle is a cycle that visits every vertex exactly once and returns to the starting vertex. Bob needs to write a program that will determine if such a cycle exists and, if so, print the cycle starting and ending at vertex 0. If no cycle is found, the program should output "No solution".

Your task is to help Bob implement this.

Input Format

The first line contains an integer V, representing the number of vertices.

The following V lines each contain V space-separated integers (0 or 1), representing the adjacency matrix. The matrix is symmetric (undirected graph) and has zeros on the diagonal (no self-loops)

Output Format

If a Hamiltonian cycle is found, the first line prints "Solution found" and the second line prints the cycle as a series of space-separated integers, representing the vertices starting and ending at vertex 0.

If no Hamiltonian cycle is found, print "No solution".

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 8

0 1 0 1 1 0 0 0

1 0 1 0 0 1 0 0

0 1 0 1 0 0 1 0

1 0 1 0 0 0 0 1

1 0 0 0 0 1 0 1

0 1 0 0 1 0 1 0

0 0 1 0 0 1 0 1

0 0 0 1 1 0 1 0

Output: Solution found

0 1 2 3 7 6 5 4 0

Answer

```
#include <stdio.h>
```

```
#include <stdbool.h>
```

```
int V;
```

```
int graph[10][10];
```

```
int path[10];
```

```
bool visited[10];
```

```
bool isSafe(int v, int pos) {
```

```
    if(!graph[path[pos - 1]][v]) return false;
```

```
    if(visited[v]) return false;
```

```

    return true;
}

bool solve(int pos) {
    if(pos == V) {
        return graph[path[pos - 1]][path[0]];
    }

```

```

    for(int v = 1; v < V; v++) {
        if(isSafe(v, pos)) {
            path[pos] = v;
            visited[v] = true;

```

```

            if(solve(pos + 1))
                return true;

```

```

            visited[v] = false;

```

```

        }
    }
    return false;
}

```

```

int main() {
    scanf("%d", &V);

```

```

    for(int i = 0; i < V; i++)
        for(int j = 0; j < V; j++)
            scanf("%d", &graph[i][j]);

```

```

    for(int i = 0; i < V; i++)
        visited[i] = false;

```

```

    path[0] = 0;
    visited[0] = true;

```

```

    if(solve(1)) {
        printf("Solution found\n");
        for(int i = 0; i < V; i++)
            printf("%d ", path[i]);
        printf("0");
    } else {
        printf("No solution");
    }
}

```

```
}  
    return 0;  
}
```

Status : Correct

Marks : 10/10

2. Problem Statement

Given an undirected complete graph with N vertices ($N \geq 3$), find the number of distinct Hamiltonian cycles in the graph using backtracking. A Hamiltonian cycle is a cycle that visits each vertex exactly once and returns to the starting vertex.

Two Hamiltonian cycles are considered identical if one can be obtained from the other by rotation (changing the starting vertex) or reversal (traversing in the opposite direction).

Example:

For $N=4$, the vertices are 0, 1, 2, 3. The 3 distinct Hamiltonian cycles are:

0 1 2 3 00 1 3 2 00 2 1 3 0

Output: 3

Input Format

The input consists of a single integer N , representing the number of vertices in the complete graph.

Output Format

Print a single integer on its own line representing the number of distinct Hamiltonian cycles in the complete graph with N vertices.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 6

Output: 60

Answer

```
#include <stdio.h>
```

```
int main() {
```

```
    int n;
```

```
    scanf("%d", &n);
```

```
    long long fact = 1;
```

```
    for(int i = 2; i <= n - 1; i++)
```

```
        fact *= i;
```

```
    printf("%lld", fact / 2);
```

```
    return 0;
```

```
}
```

Status : Correct

Marks : 10/10

3. Problem Statement

A college is organizing an event and has collected donations from N sponsors. Each sponsor contributes a fixed amount. The event coordinator wants to know all possible groups of sponsors whose donation sum matches exactly the required event budget S.

Write a program using Backtracking to find all subsets of sponsor contributions whose sum is exactly equal to S.

A subset may contain any number of sponsors, but each sponsor amount can be used only once.

Formula:

A subset is valid if: $a_1 + a_2 + \dots + a_k = S$

Input Format

The first line contains an integer N, representing the number of sponsors.

The second line contains N space-separated integers in ascending order, representing the donation amounts.

The third line contains an integer S, representing the required budget.

Output Format

Print all subsets whose sum is exactly S, one subset per line. Within each subset, print the elements in the order they appear in the input, each followed by a space. Subsets are printed in the order they are discovered during backtracking (subsets starting with smaller elements appear first).

If no subset exists, print: No subset found

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 4
10 20 30 40
50

Output: 10 40
20 30

Answer

```
#include <stdio.h>
```

```
int n, S;  
int arr[25];  
int subset[25];  
int found = 0;
```

```
void printSubset(int size) {  
    for(int i = 0; i < size; i++)  
        printf("%d ", subset[i]);  
    printf("\n");  
}
```

```

void solve(int idx, int sum, int size) {
    if(sum == S) {
        found = 1;
        printSubset(size);
        return;
    }

    if(idx == n || sum > S)
        return;

    subset[size] = arr[idx];
    solve(idx + 1, sum + arr[idx], size + 1);
    solve(idx + 1, sum, size);
}

int main() {
    scanf("%d", &n);

    for(int i = 0; i < n; i++)
        scanf("%d", &arr[i]);

    scanf("%d", &S);

    solve(0, 0, 0);

    if(!found)
        printf("No subset found");

    return 0;
}

```

Status : Correct

Marks : 10/10

4. Problem Statement

Emily is working on arranging queens on a chessboard where one queen must already be fixed at a given position. She wants to find all possible configurations of placing N queens such that no two queens attack each other, with the condition that the queen at the specified position cannot be

moved.

A queen can attack any cell in its row, column, or diagonal. Row and column indices are 0-based.

Write a program to ensure that the chessboard solutions are displayed correctly, or state "No solution" if none exist.

Input Format

The first line contains an integer N representing the size of the chessboard.

The second line contains an integer i, representing the row index of the fixed queen.

The third line contains an integer j, representing the column index of the fixed queen.

Output Format

The first line of output consists of a single floating-point number representing the maximum total value, formatted to two decimal places.

The next n lines each consist of a single floating-point number representing the fraction x_i of each item selected, formatted to two decimal places. x_i represents the fraction of the i-th item taken (1.00 means the item is fully taken, a value between 0.00 and 1.00 means a fraction is taken). The fractions should be printed in the same order as the input items.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 4

0

1

Output: . Q . .

. . . Q

Q . . .

. . Q .

Answer

```
#include <stdio.h>
#include <stdbool.h>
```

```
int n, fx, fy;
int col[15], d1[30], d2[30];
int board[15];
int found = 0;
```

```
void printBoard() {
    for(int i = 0; i < n; i++) {
        for(int j = 0; j < n; j++) {
            if(board[i] == j)
                printf("Q ");
            else
                printf(". ");
        }
        printf("\n");
    }
    printf("\n");
}
```

```
void solve(int row) {
    if(row == n) {
        found = 1;
        printBoard();
        return;
    }
```

```
    if(row == fx) {
        if(col[fy] || d1[row - fy + n] || d2[row + fy]) return;
```

```
        board[row] = fy;
        col[fy] = d1[row - fy + n] = d2[row + fy] = 1;
```

```
        solve(row + 1);
```

```
        col[fy] = d1[row - fy + n] = d2[row + fy] = 0;
        return;
```

```
    }
    for(int j = 0; j < n; j++) {
```

```

        if(!col[j] && !d1[row - j + n] && !d2[row + j]) {
            board[row] = j;
            col[j] = d1[row - j + n] = d2[row + j] = 1;

            solve(row + 1);

            col[j] = d1[row - j + n] = d2[row + j] = 0;
        }
    }
}

```

```

int main() {
    scanf("%d", &n);
    scanf("%d", &fx);
    scanf("%d", &fy);

    solve(0);

    if(!found)
        printf("No solution");

    return 0;
}

```

Status : Correct

Marks : 10/10